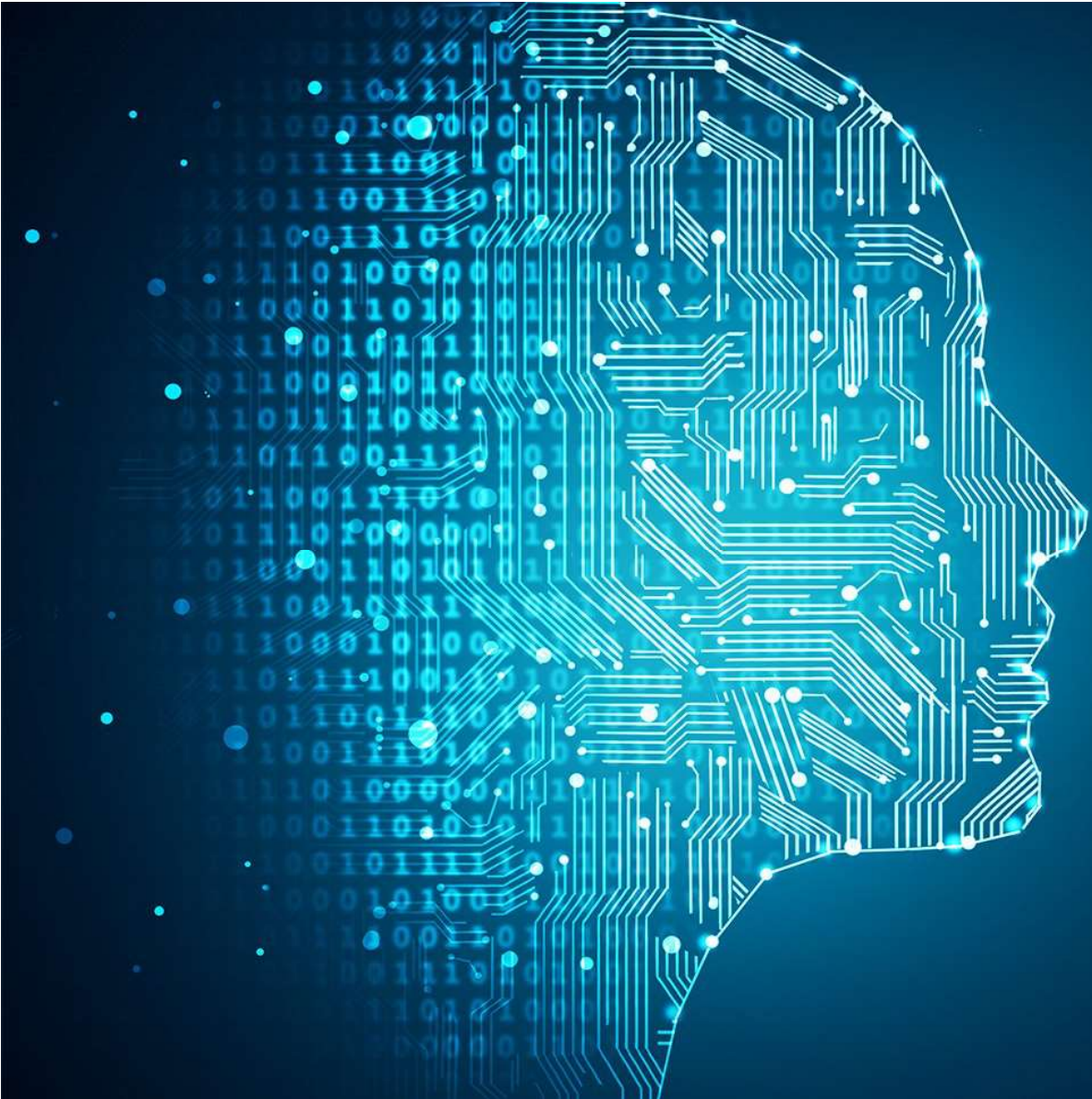




Welcome to

AASD 4014 – DL II

Instructor: Dr. Moe Fadaee

School of Computer Technology
George Brown College

Agenda:

1. Generative Adversarial Networks

Generative Adversarial Networks (GAN):

Generative Adversarial Networks were first introduced by Goodfellow et al.

Generative Adversarial Networks, 2014:

<https://arxiv.org/abs/1406.2661>

“We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake...”

Generative Adversarial Networks (GAN):

Generator:

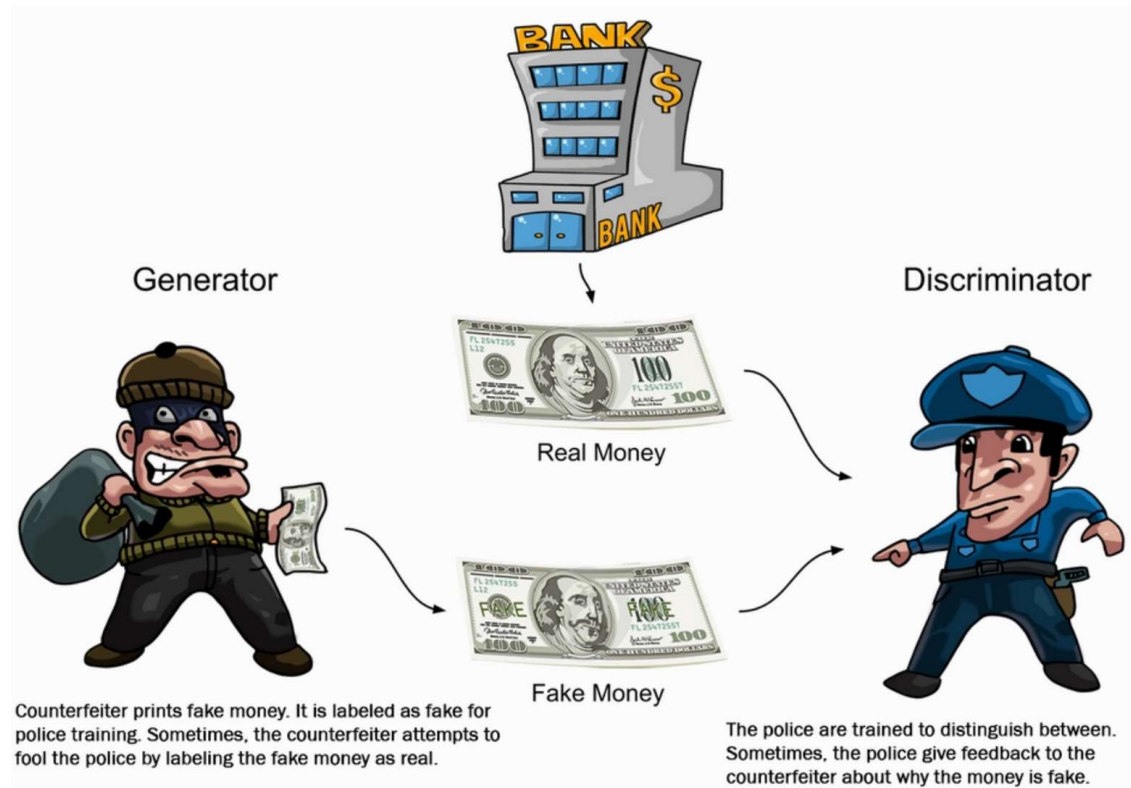
The role of generator is to create data in such a way that it can fool the discriminator.

The generated instances become negative training examples for the discriminator.

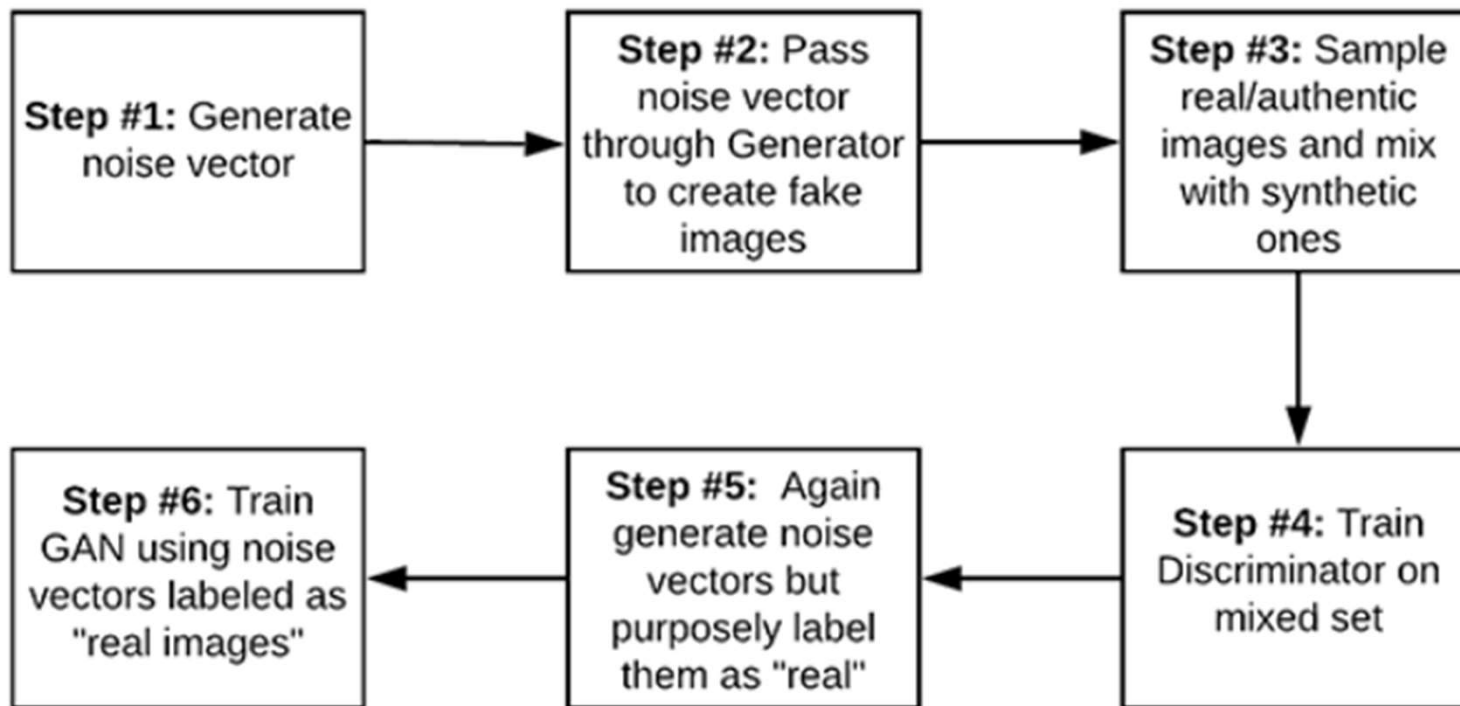
Discriminator:

The role of discriminator is to distinguish between different kind of data (i.e. the actual and generated/fake data).

The discriminator penalizes the generator for producing implausible results.



Generative Adversarial Networks (GAN): the training process



Generative Adversarial Networks (GAN): the training process

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

(1)

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

(2)

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Generative Adversarial Networks (GAN): the loss function

D = Discriminator

G = Generator

θ_d = Parameters of discriminators

θ_g = Parameters of generator

$P_z(z)$ = Input noise distribution

$P_{data}(x)$ = Original data distribution

$P_g(x)$ = Generated distribution

The binary cross-entropy loss:

$$L(\hat{y}, y) = [y \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Generative Adversarial Networks (GAN): the loss function

Now, when training the discriminator the label is $y=1$, real data and $\hat{y} = D(x)$:

$$L(D(x), 1) = \log(D(x))$$

When training the generator, the label is $y=0$, fake data and $\hat{y} = D(G(z))$:

$$L(D(G(z)), 0) = \log(1 - D(G(z)))$$

The discriminator should be able to correctly identifies the classes of the database, so the two above equations should be maximized, so final loss for Discriminator can be defined as:

$$L^D = \max[\log(D(x)) + \log(1 - D(G(z)))]$$

The generator is competing against the discriminator, G loss:

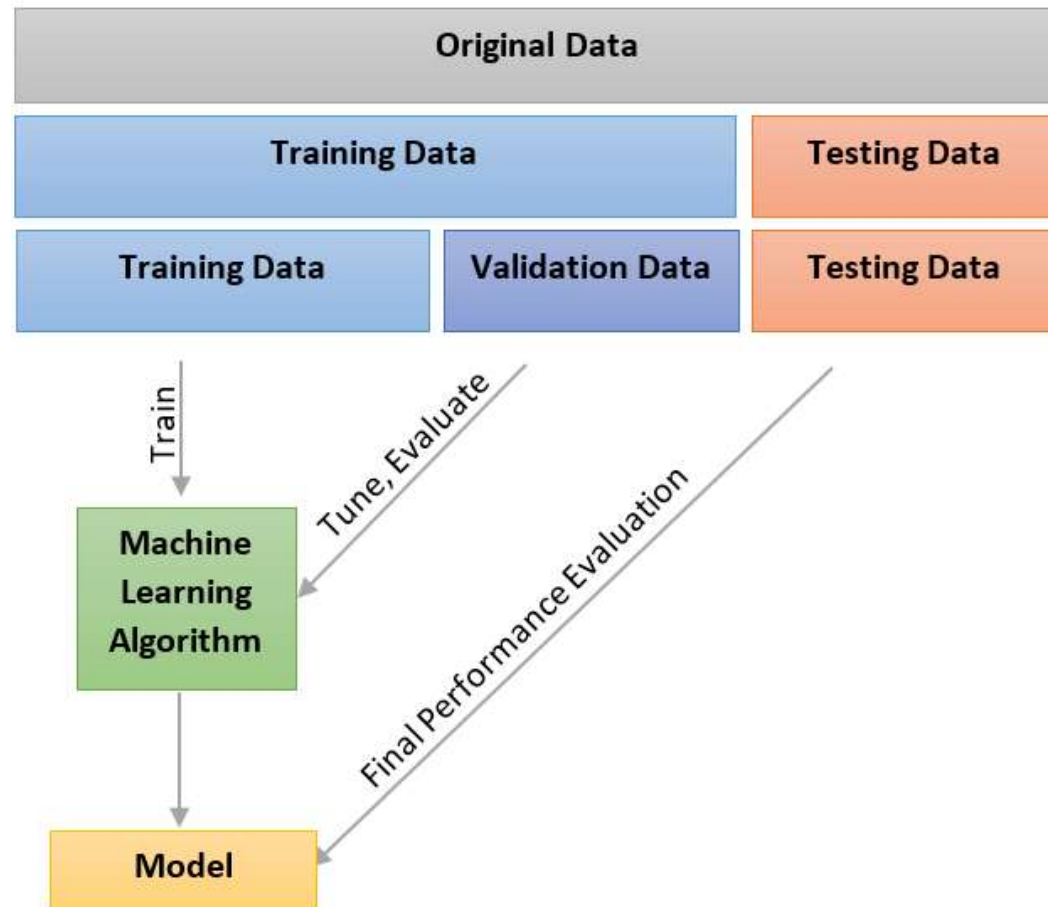
$$L^G = \min[\log(D(x)) + \log(1 - D(G(z)))]$$

Combine the 2 equations:

$$L = \min_G \max_D [\log(D(x)) + \log(1 - D(G(z)))]$$

Training vs Test sets:

- 1) Split dataset into Train and test sets
- 2) Train model on the training set
- 3) Tune model parameters on Validation set
- 4) Perform final evaluation on test set





Python:

Scikit learn: Linear Regression

For almost every model we do, we follow these:



Sklearn docs:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

Python:



Scikit learn:

A free library in Python that has great features for clustering, classification, regression and dimensionality reduction.

